

Master Presentation Script & Speaker Handout Package

Course: CSIT805: Cloud Infrastructure and Services (MCA III)

Topic: Leveraging NVIDIA H100 on Lightning AI for High-Performance Cloud Computing

Group: Group 1

Master Run of Show & Time Allocation (45 Minutes)

Time Window	Duration	Speaker	Section	Slides Covered
00:00 - 04:00	4.0 min	Ameer Hamza	Introduction, The AI Compute Wall & 45-Min Roadmap	Slide 1, Slide 2
04:00 - 07:00	3.0 min	Suhail Alam	Part 1: Why CPUs Fail at AI & The On-Premises Barrier	Slide 3, Slide 4
07:00 - 15:30	8.5 min	Harsh Mishra	Part 2: NVIDIA Hopper H100 Silicon Deep-Dive	Slide 5, 6, 7, 8
15:30 - 23:00	7.5 min	Mirza Saad Beg	Part 3: Cloud Infrastructure, Elasticity & Distributed Scaling	Slide 9, 10, 11
23:00 - 36:00	13.0 min	Ameer Hamza	Part 4: LIVE Supercomputer Hardware Showdown & Telemetry	Slide 12, Terminal, Slide 13
36:00 - 41:00	5.0 min	Harshit Tandon	Part 5: Hyperscaler Quota Bottlenecks & Cloud Economics	Slide 14, Slide 15
41:00 - 45:00	4.0 min	Ameer & Team	Core Conclusions & Faculty Viva Defense	Slide 16, Slide 17

PART I: MASTER SLIDE-BY-SLIDE SPOKEN SCRIPT

[SLIDE 1: Title Slide & Partner Ecosystem]

Speaker: Ameer Hamza

Timing: 00:00 - 02:00

"Good morning, respected professors and fellow classmates. Group 1 presents our study: **Leveraging NVIDIA H100 on Lightning AI for High-Performance Cloud Computing: Bridging the Gap Between Consumer Hardware and Enterprise Supercomputing for Modern AI Workloads.**

Between 2012 and 2026, the computing power needed to train modern AI models grew by more than **100 million times**. For decades, computer science relied on CPUs getting faster every year under Moore's Law. But CPUs have hit a hard physical wall: they generate too much heat if pushed any faster. Trying to train a modern 70-billion-parameter language model on standard server CPUs would take decades and huge amounts of electricity.

To solve this, modern AI relies on specialized chips: specifically the **NVIDIA Hopper H100 GPU**, combined with modern cloud platforms like **Lightning AI** that let us spin up an 80 GB supercomputer in under 30 seconds."

[SLIDE 2: 45-Minute Roadmap & Team Architecture]

Speaker: Ameer Hamza

Timing: 02:00 - 04:00

"Our presentation is divided into five clear engineering parts:

1. **Suhail Alam** will explain why CPUs cannot handle deep learning math, and why buying your own physical AI supercomputer is practically impossible.
2. **Harsh Mishra** will take us inside the H100 chip: covering its 80 billion transistors, the new FP8 Transformer Engine, and ultra-fast 3.35 TB/s HBM3 memory.
3. **Mirza Saad Beg** will cover the cloud layer: how Lightning AI removes setup headaches, switches hardware on the fly, and runs large models across multiple GPUs using FSDP.
4. **I will then run a live demo on an active cloud H100:** running four live tests comparing our laptop's RTX 4060 against the H100 on memory limits, fine-tuning a 3-billion-parameter model, real-time chat, and handling 16 users at once.
5. **Harshit Tandon** will break down the cloud costs: comparing AWS and Google Cloud quota hurdles against Lightning AI's pay-as-you-go pricing.
6. Finally, we will open the floor for your questions.

To start with why CPUs struggle with AI math, I hand over to Suhail Alam."

[SLIDE 3: Why CPUs Fail at Modern AI Workloads]

Speaker: Suhail Alam

Timing: 04:00 - 05:30

"Thank you, Ameer. Let us start with a foundational question: **Why can't our powerful Intel or AMD server CPUs train modern AI models?**

It comes down to how they are designed: – A modern CPU is built for **speed on sequential tasks**. It has 16 to 64 large cores optimized for complex logic, step-by-step instructions, and operating systems. – Deep learning is completely different. It is not complex branching logic; it is **huge amounts of matrix math**—trillions of multiplications and additions happening all at the same time.

Think of a CPU like a fast sports car that can carry only a few passengers very quickly. A GPU is like a **10,000-lane highway**. Even if each lane moves at normal speed, millions of data points move forward together in every single clock cycle. While a CPU computes matrix numbers one by one in a loop, a modern GPU crunches entire matrix blocks in parallel."

[SLIDE 4: The On-Premises Impossibility & The Cloud Shift]

Speaker: Suhail Alam

Timing: 05:30 - 07:00

"If GPUs are essential, why doesn't every college lab or startup just buy an NVIDIA H100 server?

Because physical AI supercomputers come with three major real-world blockers:

1. **Purchase Cost (CapEx):** A single server with 8 H100 GPUs costs over **\$300,000**—that is more than 2.5 Crore Indian Rupees.
2. **Power Draw:** That single box pulls **10.2 Kilowatts of continuous electricity**. Standard college lab wiring cannot support this load without dedicated industrial power lines.
3. **Heat & Cooling:** Each H100 chip outputs **700 Watts of heat**. Normal air conditioning cannot cool it; it requires specialized industrial liquid cooling.

For most teams, buying the physical hardware makes no financial sense. The cloud solves this by turning a multi-crore machine into a simple utility service: **renting an 80 GB H100 for about \$3.00 an hour on demand**.

Now, let us look at what makes the H100 chip itself so fast. Harsh Mishra will walk us through its architecture."

[SLIDE 5: NVIDIA Hopper GH100 Architecture]

Speaker: Harsh Mishra

Timing: 07:00 - 09:00

"Thank you, Suhail. Respected faculty, let us look at the silicon engineering inside the **NVIDIA Hopper H100**.

Built on a custom **4-nanometer process node by TSMC**, the chip packs **80 Billion transistors** onto a single silicon die of **814 square millimeters**—which is the maximum size chip factories can physically produce today.

Inside the cloud SXM5 version, you get: - **132 Streaming Multiprocessors (SMs)**. - **16,896 CUDA Cores** for general floating-point math. - **528 4th-Generation Tensor Cores**, delivering up to **3,000 TeraFLOPS** of AI compute. - A huge **50 Megabyte L2 Cache**, keeping data close to the cores so they don't waste time waiting for external memory. - Hardware support for **Distributed Shared Memory**, allowing execution blocks across different SMs to share data directly without going back to main memory.

This physical hardware powers three major features: the FP8 Transformer Engine, HBM3 memory, and NVLink 4."

[SLIDE 6: The FP8 Transformer Engine Revolution]

Speaker: Harsh Mishra

Timing: 09:00 - 11:00

"The first major breakthrough in Hopper is the **FP8 Transformer Engine**.

Traditionally, AI models were trained using **FP32** (32-bit numbers) or **FP16** (16-bit numbers). Here is why number size matters: - A 16-bit number takes 2 bytes of storage. - An **8-bit number (FP8)** takes only **1 byte of storage**. - Cutting the size in half means you move half as much data across memory, and your Tensor Cores can process calculations **twice as fast**.

The catch is that simply cutting numbers to 8 bits can lose critical precision and cause training to fail.

Hopper fixes this automatically in hardware: 1. The Transformer Engine constantly monitors value ranges during training. 2. It uses **E4M3** (more precision) during the forward activation pass. 3. It switches to **E5M2** (wider dynamic range) during the backward gradient pass. 4. It scales values on the fly so small gradient numbers do not round down to zero.

This gives us **2x to 3x faster training speed with zero loss in final model accuracy**."

[SLIDE 7: Overcoming the Memory Wall: HBM3 Subsystem]

Speaker: Harsh Mishra

Timing: 11:00 - 13:00

"The second breakthrough solves the biggest bottleneck in computer architecture: **The Memory Wall**.

In deep learning, raw compute power is useless if your cores sit idle waiting for data to arrive from memory. In transformer models, generating tokens word-by-word is strictly **memory-bandwidth bound**.

Look at the memory comparison on Slide 7: - Standard DDR5 computer memory moves data at about **64 GB/s**. - An RTX 4060 laptop GPU moves data at **272 GB/s** over a 128-bit bus. - The NVIDIA H100 SXM5 does something completely different: it stacks 3D memory chips directly next to the GPU die using tiny vertical connections called Through-Silicon Vias. This creates a massive **5,120-bit wide bus** called **HBM3**.

This delivers **3.35 Terabytes per second (3,350 GB/s)** of memory bandwidth. That is **12.3 times faster than our laptop GPU**, ensuring the compute cores never run out of data."

[SLIDE 8: NVLink 4 & NVSwitch: Multi-GPU Mesh]

Speaker: Harsh Mishra

Timing: 13:00 - 15:30

"When a model has 70 billion parameters, it cannot fit inside one 80 GB card. You must connect multiple GPUs together.

In standard servers, graphics cards talk over PCIe slots. Modern **PCIe Gen 5** tops out at **64 GB/s**. When multiple GPUs try to share billions of numbers during training, PCIe becomes a massive traffic jam.

NVIDIA solved this with **NVLink 4**: - Each H100 has **18 NVLink connections**, giving **900 GB/s** of speed between GPUs—**14 times faster than PCIe Gen 5**. - Combined with **NVSwitch** chips on the motherboard, all 8 GPUs in a server talk to each other at full speed.

To software, an 8-GPU server does not look like eight separate cards; it behaves like a **single, unified 640 GB supercomputer**.

Now, how do engineers actually use this hardware in the cloud without getting stuck in complex devops setups? Mirza Saad Beg will explain the cloud systems layer."

[SLIDE 9: The Cloud Abstraction Problem & Lightning AI]

Speaker: Mirza Saad Beg

Timing: 15:30 - 17:30

"Thank you, Harsh. Having powerful GPUs is great, but in traditional cloud setups like standard AWS or Google Cloud, getting them running is often painful: - Engineers spend hours resolving Linux driver mismatches, CUDA version errors, and Docker containers. - You have to write complex Kubernetes configuration files and security rules. - And if you accidentally leave an idle GPU running over the weekend, your company gets a bill for hundreds of dollars.

Lightning AI eliminates these setup headaches.

It provides **Cloud Studios**—browser-based environments that launch in under 30 seconds. You get ready-to-use CUDA 12.1 drivers, persistent disk storage, and simple hardware selection, so you can write normal PyTorch code without dealing with cloud devops."

[SLIDE 10: Dynamic Cloud Elasticity]

Speaker: Mirza Saad Beg

Timing: 17:30 - 19:30

"The most useful feature of Lightning AI is **Dynamic Hardware Elasticity**.

On standard cloud servers, if you want to switch from a cheap CPU to an H100, you have to stop the virtual machine, detach disks, create a new server, and set up everything again.

In Lightning AI, it takes one click: - You write your code and prepare your data on a **Free 4-Core CPU at zero cost**. - When you are ready to train, you open the hardware menu and choose **NVIDIA H100**. - In **under 30 seconds**, Lightning AI moves your workspace over to an active 80 GB H100 node—keeping all your files, open terminal tabs, and code completely intact. - When training finishes, you switch back to Free CPU. You pay only for the exact minutes the GPU was running."

[SLIDE 11: Distributed AI: DDP vs FSDP]

Speaker: Mirza Saad Beg

Timing: 19:30 - 23:00

"When training larger models across multiple GPUs, engineers use two common techniques:

1. DDP (Distributed Data Parallel):

- The entire model is copied onto every GPU.
- Each GPU takes a different chunk of data and computes updates.
- At the end of each step, the GPUs sync their results over fast NVLink lines.
- This is fast and simple, but the entire model must fit inside a single 80 GB GPU.

2. FSDP (Fully Sharded Data Parallel):

- When a model has 70B or 400B parameters, it is too big for a single 80 GB card.
- FSDP splits the model weights, gradients, and optimizer across all GPUs in the cluster.
- Each GPU holds only a fraction of the model in memory. Layers are fetched right when needed over fast NVLink lines and then freed immediately.
- This lets teams train massive models without running out of memory.

In Lightning AI, you can scale from one GPU to multi-GPU FSDP without rewriting your code.

Now, let us see the real-world proof. I hand over to Ameer Hamza for our live demo."

[SLIDE 12: The Hardware Showdown — Live Supercomputer Demonstration]

Speaker: Ameer Hamza

Timing: 23:00 - 24:30

"Thank you, Saad. Welcome to our live hardware showdown: - **Team Red (Our Laptop Underdog):** An **RTX 4060 Laptop GPU** with 8 GB GDDR6 memory, a 128-bit bus, and 272 GB/s bandwidth. - **Team Green (The Cloud Supercomputer):** A **Cloud NVIDIA H100** on Lightning AI with 80 GB HBM3 memory, a 5,120-bit bus, and 3,350 GB/s bandwidth.

Notice the visual memory gauges on Slide 12: - On the laptop, trying to run enterprise AI immediately hits the **8 GB hard limit**, causing an out-of-memory crash. - On the H100, that exact same workload takes only **15% of memory (12 GB out of 80 GB)**, leaving 68 GB completely free.

Let us switch directly to our active terminal in our live Lightning Studio to run our four benchmark scripts."

[LIVE DEMONSTRATION SCRIPT — RUNNING IN ACTIVE TERMINAL]

Speaker: Ameer Hamza

Timing: 24:30 - 32:00

Step 0: Hardware Telemetry Verification

"Let us first inspect our cloud node:

```
nvidia-smi
```

Look at the terminal output: - **Product Name:** NVIDIA H100 80GB HBM3. - **Driver Version:** CUDA Version 12.x ready. - **Physical Memory:** 81,559 Megabytes (80 GB). - **Power Limit:** 700 Watts maximum capacity. This confirms our direct connection to Hopper silicon."

Step 1: Experiment 1 — The VRAM Capacity Wall

```
python hands_on_demo/exp1_vram_oom.py
```

"In Experiment 1, we try to allocate a **12.00 Gigabyte FP32 tensor**, simulating a batch size of 64 on an attention layer.

On our RTX 4060 laptop, PyTorch fails immediately with `RuntimeError: CUDA out of memory` because 12 GB cannot fit inside 8 GB of VRAM.

Now look at our Cloud H100: - The allocation succeeds in **144.11 milliseconds**. - The script reports: **67.18 Gigabytes of VRAM remaining completely free**. Where consumer hardware crashes, the cloud supercomputer runs effortlessly."

Step 2: Experiment 2 — Real LLM Fine-Tuning (Qwen-2.5 3B)

```
python hands_on_demo/exp2_real_llm_finetune.py --epochs 1
```

"In Experiment 2, we run real fine-tuning: training **Qwen-2.5 3B** on an entire epoch of **332 question-answer pairs** using LoRA.

On our RTX 4060 laptop: - Physical 8GB memory limits us strictly to **Batch Size 1**. - The laptop must execute **332 separate steps**, running at **5.4 samples per second**, and taking **64 seconds**.

Now look at the exact same command on the Cloud H100: - With 80 GB HBM3 memory, the H100 uses **Batch Size 16**. - The entire epoch finishes in only **21 parallel steps**. - Total time: **2.3 seconds** at **147.1 samples per second**. That is a measured **27.8x speedup** on the exact same dataset and model."

Step 3: Experiment 3 — Interactive Real-Time Token Streaming

```
python hands_on_demo/exp3_inference_speed.py
```

"In Experiment 3, we test real-time chat speed using our fine-tuned model. We ask it: `'What is the minimum attendance criteria at Amity University?'`

Look at the response stream: - Time to the first token is under **120 milliseconds**. - Generation streams smoothly at over **180 tokens per second**, answering with the exact 75% attendance rule from our dataset. This shows instant interactive speed."

Step 4: Experiment 4 — 16-User Concurrent Enterprise Serving Stress Test

```
python hands_on_demo/exp4_enterprise_stress_test.py
```

"In Experiment 4, we simulate production cloud serving: **16 users asking technical questions at the exact same second.**

On the RTX 4060 laptop with its narrow 128-bit memory bus, parallel batching fails. Queries must queue up one by one, taking **25.7 seconds.**

Now look at the H100: - The H100 processes all 16 prompt streams together across its 5,120-bit HBM3 bus.
- All 16 queries finish in **1.35 seconds.** - Total throughput: **1,480+ tokens per second**—a **19.7x concurrency advantage** over laptop hardware."

[SLIDE 13: Measured Benchmark Matrix]

Speaker: Ameer Hamza

Timing: 32:00 - 36:00

"Slide 13 summarizes these measured results:

1. **Memory Pipeline:** 192 GB/s on RTX 3050, 272 GB/s on RTX 4060, versus **3,350 GB/s on H100** (12.3x faster data delivery).
2. **Fine-Tuning Speed:** 64 seconds on the laptop versus **2.3 seconds on the H100** (27.8x speedup).
3. **Multi-User Serving:** 25.7 seconds on the laptop versus **1.35 seconds on the H100** (19.7x concurrency speedup).

The conclusion is simple: consumer GPUs below 4060 crash immediately; the 4060 can only process one sample at a time; enterprise AI demands high-bandwidth cloud GPUs.

To explain cloud pricing and access barriers, here is Harshit Tandon."

[SLIDE 14: The Hyperscaler Bottleneck vs Lightning AI]

Speaker: Harshit Tandon

Timing: 36:00 - 38:30

"Thank you, Ameer. Let us look at the business and pricing side: **How you actually get access to an H100.**

If you try to rent an H100 on traditional clouds like **Amazon Web Services (AWS)** or **Google Cloud Platform (GCP)**, you immediately run into the **Quota Bottleneck:**

1. **Default Zero Limit:** Your account starts with zero GPU quota. You cannot launch a machine without submitting justification requests.
2. **Forced 8-GPU Bundles:** AWS and GCP do not rent single H100s. They force you to rent an entire 8-GPU box at **\$88 to \$98 per hour.**
3. **Long Contracts:** Sales teams often require 1-year or 3-year commitments of \$50,000 to \$100,000 upfront.
4. **Delays:** For students or small teams, approval takes days or weeks and is often rejected.

Lightning AI makes access simple: - You can rent a **single H100 GPU for about \$3.00 an hour.** - There are no sales calls, no quota forms, and no annual contracts. - Any student or startup can start an H100 in 30 seconds."

[SLIDE 15: Enterprise TCO & Cost Optimization]

Speaker: Harshit Tandon

Timing: 38:30 - 41:00

"Now look at the total cost comparison on Slide 15:

- **The Wasteful Cloud Approach:** A team starts an 8-GPU node on AWS and leaves it running all month. That costs **\$21,600 per month**. Studies show up to 60% of that GPU time sits completely idle while developers write code or fix bugs.
- **The Smart Studio Approach:** With Lightning AI, you match the hardware to the task:
 - Writing code and testing on **Free CPU: \$0.00**.
 - Training on an H100 for 10 minutes: **\$0.50**.
 - Keeping files in cloud storage: **a few cents per month**.

Notice the live demo Ameer just ran: - All four live tests ran in **less than 3 minutes of GPU time**. - The total bill was **under 15 cents—about 12 Indian Rupees!**

This turns AI supercomputing into an affordable service that any student or startup can use.

I invite Ameer back to wrap up our conclusions and start our viva."

[SLIDE 16: Conclusion: The New AI Paradigm]

Speaker: Ameer Hamza

Timing: 41:00 - 42:30

"Thank you, Harshit. To summarize our project into four takeaways:

1. **Specialized Hardware is Mandatory:** CPUs cannot keep up with AI math. Modern workloads require Tensor Cores, FP8 math, and HBM3 memory.
 2. **Memory Speed Matters Most:** Fast compute is useless without fast memory. The H100's **3,350 GB/s HBM3 memory** delivers 19.7x faster multi-user serving than laptop GDDR6.
 3. **Cloud Simplicity Wins:** Lightning AI turns complex cloud setups into 30-second ready-to-use studios.
 4. **Affordable Access:** Renting a single GPU for **\$3.00/hour** gives students supercomputing power without buying a \$300,000 server."
-

[SLIDE 17: Questions & Faculty Viva Defense]

Speaker: Ameer Hamza & All Team Members

Timing: 42:30 - 45:00

"Respected faculty evaluators and professors, Group 1 concludes our presentation. Our code, test scripts, trained weights, and presentation slides are available on our GitHub repository.

Each member is ready for your questions: - **Ameer Hamza:** Demo scripts, benchmarking, and throughput numbers. - **Suhail Alam:** CPU limits and physical server data center challenges. - **Harsh Mishra:** H100 chip design, FP8 math, and HBM3 memory. - **Mirza Saad Beg:** Cloud infrastructure, elasticity, and DDP vs FSDP. - **Harshit Tandon:** Cloud pricing, quota limits, and total cost of ownership.

We welcome your questions."

PART II: INDIVIDUAL PRINTABLE SPEAKER PACKETS

SPEAKER PACKET 1: AMEER HAMZA

Role: Host, Live Demo Masterclass, Executive Synthesis, Faculty Defense

Assigned Slides: Slide 1, Slide 2, Slide 12, Terminal Live Demo, Slide 13, Slide 16, Slide 17

Total Speaking Time: ~17.0 Minutes

Slide 1 (Title Slide & Partner Ecosystem)

"Good morning, respected professors and fellow classmates. Group 1 presents our study: **Leveraging NVIDIA H100 on Lightning AI for High-Performance Cloud Computing: Bridging the Gap Between Consumer Hardware and Enterprise Supercomputing for Modern AI Workloads.**

Between 2012 and 2026, the computing power needed to train modern AI models grew by more than **100 million times**. For decades, computer science relied on CPUs getting faster every year under Moore's Law. But CPUs have hit a hard physical wall: they generate too much heat if pushed any faster. Trying to train a modern 70-billion-parameter language model on standard server CPUs would take decades and huge amounts of electricity.

To solve this, modern AI relies on specialized chips: specifically the **NVIDIA Hopper H100 GPU**, combined with modern cloud platforms like **Lightning AI** that let us spin up an 80 GB supercomputer in under 30 seconds."

Slide 2 (Roadmap & Team Architecture)

"Our presentation is divided into five clear engineering parts:

1. **Suhail Alam** will explain why CPUs cannot handle deep learning math, and why buying your own physical AI supercomputer is practically impossible.
2. **Harsh Mishra** will take us inside the H100 chip: covering its 80 billion transistors, the new FP8 Transformer Engine, and ultra-fast 3.35 TB/s HBM3 memory.
3. **Mirza Saad Beg** will cover the cloud layer: how Lightning AI removes setup headaches, switches hardware on the fly, and runs large models across multiple GPUs using FSDP.
4. **I will then run a live demo on an active cloud H100:** running four live tests comparing our laptop's RTX 4060 against the H100 on memory limits, fine-tuning a 3-billion-parameter model, real-time chat, and handling 16 users at once.
5. **Harshit Tandon** will break down the cloud costs: comparing AWS and Google Cloud quota hurdles against Lightning AI's pay-as-you-go pricing.
6. Finally, we will open the floor for your questions.

To start with why CPUs struggle with AI math, I hand over to Suhail Alam."

(Hand off to Suhail Alam)

Slide 12 (The Hardware Showdown Announcement)

(Take over from Mirza Saad Beg)

"Thank you, Saad. Welcome to our live hardware showdown: - **Team Red (Our Laptop Underdog)**: An **RTX 4060 Laptop GPU** with 8 GB GDDR6 memory, a 128-bit bus, and 272 GB/s bandwidth. - **Team Green (The Cloud Supercomputer)**: A **Cloud NVIDIA H100** on Lightning AI with 80 GB HBM3 memory, a 5,120-bit bus, and 3,350 GB/s bandwidth.

Notice the visual memory gauges on Slide 12: - On the laptop, trying to run enterprise AI immediately hits the **8 GB hard limit**, causing an out-of-memory crash. - On the H100, that exact same workload takes only **15% of memory (12 GB out of 80 GB)**, leaving 68 GB completely free.

Let us switch directly to our active terminal in our live Lightning Studio to run our four benchmark scripts."

Terminal Live Demo Execution

(Run commands live in terminal)

Step 0: `nvidia-smi` "Let us first inspect our cloud node: Observe the telemetry: NVIDIA H100 80GB HBM3, Driver 535 ready, 81,559 Megabytes total VRAM, and a 700W power ceiling. This confirms our direct connection to Hopper silicon."

Step 1: `python hands_on_demo/exp1_vram_oom.py` "In Experiment 1, we try to allocate a **12.00 Gigabyte FP32 tensor**, simulating a batch size of 64 on an attention layer. On our RTX 4060 laptop, PyTorch fails immediately with `RuntimeError: CUDA out of memory`. On our Cloud H100, the allocation completes in **144.11 milliseconds**, with **67.18 Gigabytes of VRAM remaining completely free.**"

Step 2: `python hands_on_demo/exp2_real_llm_finetune.py --epochs 1` "In Experiment 2, we run real fine-tuning: training **Qwen-2.5 3B** on 332 questions using LoRA. On the RTX 4060 laptop, physical 8GB memory limits us strictly to **Batch Size 1**, taking 332 steps at **5.4 samples/sec**, and finishing in **64 seconds**. On the Cloud H100, 80GB HBM3 memory allows **Batch Size 16**, completing in **21 parallel steps in just 2.3 seconds** at **147.1 samples/sec**—a measured **27.8x speedup.**"

Step 3: `python hands_on_demo/exp3_inference_speed.py` "In Experiment 3, we test real-time chat speed using our fine-tuned model. We ask: *'What is the minimum attendance criteria at Amity University?'* Time to first token is under **120 milliseconds**, streaming at over **180 tokens per second** with exact accuracy."

Step 4: `python hands_on_demo/exp4_enterprise_stress_test.py` "In Experiment 4, we test **16 concurrent enterprise queries** hitting the model simultaneously. On the laptop, queries queue serially, taking **25.7 seconds**. On the H100, all 16 queries process together across the 5,120-bit HBM3 bus, finishing in **1.35 seconds** at **1,480+ tokens per second**—a **19.7x concurrency advantage.**"

Slide 13 (Measured Benchmark Matrix)

"Slide 13 summarizes these measured results: 1. **Memory Pipeline**: 192 GB/s on 3050, 272 GB/s on 4060, versus **3,350 GB/s on H100** (12.3x faster data delivery). 2. **Fine-Tuning Speed**: 64 seconds on laptop versus **2.3 seconds on H100** (27.8x speedup). 3. **Multi-User Serving**: 25.7 seconds on laptop versus **1.35 seconds on H100** (19.7x advantage).

The conclusion is simple: consumer GPUs below 4060 crash immediately; the 4060 can only process one sample at a time; enterprise AI demands high-bandwidth cloud GPUs.

To explain cloud pricing and access barriers, here is Harshit Tandon."

(Hand off to Harshit Tandon)

Slide 16 (Executive Synthesis)

(Take over from Harshit Tandon)

"Thank you, Harshit. To summarize our project into four takeaways:

1. **Specialized Hardware is Mandatory:** CPUs cannot keep up with AI math. Modern workloads require Tensor Cores, FP8 math, and HBM3 memory.
2. **Memory Speed Matters Most:** Fast compute is useless without fast memory. The H100's **3,350 GB/s HBM3 memory** delivers 19.7x faster multi-user serving than laptop GDDR6.
3. **Cloud Simplicity Wins:** Lightning AI turns complex cloud setups into 30-second ready-to-use studios.
4. **Affordable Access:** Renting a single GPU for **\$3.00/hour** gives students supercomputing power without buying a \$300,000 server."

Slide 17 (Faculty Viva Opening)

"Respected faculty evaluators and professors, Group 1 concludes our presentation. Our code, test scripts, trained weights, and presentation slides are available on our GitHub repository. Our team is prepared for your questions. The floor is open for viva defense."

SPEAKER PACKET 2: SUHAIL ALAM

Role: Foundations of AI Compute & The On-Premises Barrier

Assigned Slides: Slide 3, Slide 4

Total Speaking Time: 3.0 Minutes (04:00 - 07:00)

Slide 3 (Why CPUs Fail at Modern AI Workloads)

(Take over from Ameer Hamza)

"Thank you, Ameer. Let us start with a foundational question: **Why can't our powerful Intel or AMD server CPUs train modern AI models?**

It comes down to how they are designed: - A modern CPU is built for **speed on sequential tasks**. It has 16 to 64 large cores optimized for complex logic, step-by-step instructions, and operating systems. - Deep learning is completely different. It is not complex branching logic; it is **huge amounts of matrix math**—trillions of multiplications and additions happening all at the same time.

Think of a CPU like a fast sports car that can carry only a few passengers very quickly. A GPU is like a **10,000-lane highway**. Even if each lane moves at normal speed, millions of data points move forward together in every single clock cycle. While a CPU computes matrix numbers one by one in a loop, a modern GPU crunches entire matrix blocks in parallel."

Slide 4 (The On-Premises Impossibility & The Cloud Shift)

"If GPUs are essential, why doesn't every college lab or startup just buy an NVIDIA H100 server?

Because physical AI supercomputers come with three major real-world blockers:

1. **Purchase Cost (CapEx):** A single server with 8 H100 GPUs costs over **\$300,000**—that is more than 2.5 Crore Indian Rupees.
2. **Power Draw:** That single box pulls **10.2 Kilowatts of continuous electricity**. Standard college lab wiring cannot support this load without dedicated industrial power lines.
3. **Heat & Cooling:** Each H100 chip outputs **700 Watts of heat**. Normal air conditioning cannot cool it; it requires specialized industrial liquid cooling.

For most teams, buying the physical hardware makes no financial sense. The cloud solves this by turning a multi-crore machine into a simple utility service: **renting an 80 GB H100 for about \$3.00 an hour on demand**.

Now, let us look at what makes the H100 chip itself so fast. Harsh Mishra will walk us through its architecture."

(Hand off to Harsh Mishra)

SPEAKER PACKET 3: HARSH MISHRA

Role: NVIDIA Hopper H100 Silicon Engineering

Assigned Slides: Slide 5, Slide 6, Slide 7, Slide 8

Total Speaking Time: 8.5 Minutes (07:00 - 15:30)

Slide 5 (NVIDIA Hopper GH100 Architecture)

(Take over from Suhail Alam)

"Thank you, Suhail. Respected faculty, let us look at the silicon engineering inside the **NVIDIA Hopper H100**.

Built on a custom **4-nanometer process node by TSMC**, the chip packs **80 Billion transistors** onto a single silicon die of **814 square millimeters**—which is the maximum size chip factories can physically produce today.

Inside the cloud SXM5 version, you get: - **132 Streaming Multiprocessors (SMs)**. - **16,896 CUDA Cores** for general floating-point math. - **528 4th-Generation Tensor Cores**, delivering up to **3,000 TeraFLOPS** of AI compute. - A huge **50 Megabyte L2 Cache**, keeping data close to the cores so they don't waste time waiting for external memory. - Hardware support for **Distributed Shared Memory**, allowing execution blocks across different SMs to share data directly without going back to main memory.

This physical hardware powers three major features: the FP8 Transformer Engine, HBM3 memory, and NVLink 4."

Slide 6 (The FP8 Transformer Engine Revolution)

"The first major breakthrough in Hopper is the **FP8 Transformer Engine**.

Traditionally, AI models were trained using **FP32** (32-bit numbers) or **FP16** (16-bit numbers). Here is why number size matters: - A 16-bit number takes 2 bytes of storage. - An **8-bit number (FP8)** takes only **1 byte of storage**. - Cutting the size in half means you move half as much data across memory, and your Tensor Cores can process calculations **twice as fast**.

The catch is that simply cutting numbers to 8 bits can lose critical precision and cause training to fail.

Hopper fixes this automatically in hardware: 1. The Transformer Engine constantly monitors value ranges during training. 2. It uses **E4M3** (more precision) during the forward activation pass. 3. It switches to **E5M2** (wider dynamic range) during the backward gradient pass. 4. It scales values on the fly so small gradient numbers do not round down to zero.

This gives us **2x to 3x faster training speed with zero loss in final model accuracy**."

Slide 7 (Overcoming the Memory Wall: HBM3 Subsystem)

"The second breakthrough solves the biggest bottleneck in computer architecture: **The Memory Wall**.

In deep learning, raw compute power is useless if your cores sit idle waiting for data to arrive from memory. In transformer models, generating tokens word-by-word is strictly **memory-bandwidth bound**.

Look at the memory comparison on Slide 7: - Standard DDR5 computer memory moves data at about **64 GB/s**. - An RTX 4060 laptop GPU moves data at **272 GB/s** over a 128-bit bus. - The NVIDIA H100 SXM5 does something completely different: it stacks 3D memory chips directly next to the GPU die using tiny vertical connections called Through-Silicon Vias. This creates a massive **5,120-bit wide bus** called **HBM3**.

This delivers **3.35 Terabytes per second (3,350 GB/s)** of memory bandwidth. That is **12.3 times faster than our laptop GPU**, ensuring the compute cores never run out of data."

Slide 8 (NVLink 4 & NVSwitch: Multi-GPU Mesh)

"When a model has 70 billion parameters, it cannot fit inside one 80 GB card. You must connect multiple GPUs together.

In standard servers, graphics cards talk over PCIe slots. Modern **PCIe Gen 5** tops out at **64 GB/s**. When multiple GPUs try to share billions of numbers during training, PCIe becomes a massive traffic jam.

NVIDIA solved this with **NVLink 4**: - Each H100 has **18 NVLink connections**, giving **900 GB/s** of speed between GPUs—**14 times faster than PCIe Gen 5**. - Combined with **NVSwitch** chips on the motherboard, all 8 GPUs in a server talk to each other at full speed.

To software, an 8-GPU server does not look like eight separate cards; it behaves like a **single, unified 640 GB supercomputer**.

Now, how do engineers actually use this hardware in the cloud without getting stuck in complex devops setups? Mirza Saad Beg will explain the cloud systems layer."

(Hand off to Mirza Saad Beg)

SPEAKER PACKET 4: MIRZA SAAD BEG

Role: Cloud Infrastructure, Virtualization & Distributed Scaling

Assigned Slides: Slide 9, Slide 10, Slide 11

Total Speaking Time: 7.5 Minutes (15:30 - 23:00)

Slide 9 (The Cloud Abstraction Problem & Lightning AI)

(Take over from Harsh Mishra)

“Thank you, Harsh. Having powerful GPUs is great, but in traditional cloud setups like standard AWS or Google Cloud, getting them running is often painful: - Engineers spend hours resolving Linux driver mismatches, CUDA version errors, and Docker containers. - You have to write complex Kubernetes configuration files and security rules. - And if you accidentally leave an idle GPU running over the weekend, your company gets a bill for hundreds of dollars.

Lightning AI eliminates these setup headaches.

It provides **Cloud Studios**—browser-based environments that launch in under 30 seconds. You get ready-to-use CUDA 12.1 drivers, persistent disk storage, and simple hardware selection, so you can write normal PyTorch code without dealing with cloud devops.”

Slide 10 (Dynamic Cloud Elasticity)

“The most useful feature of Lightning AI is **Dynamic Hardware Elasticity**.

On standard cloud servers, if you want to switch from a cheap CPU to an H100, you have to stop the virtual machine, detach disks, create a new server, and set up everything again.

In Lightning AI, it takes one click: - You write your code and prepare your data on a **Free 4-Core CPU at zero cost**. - When you are ready to train, you open the hardware menu and choose **NVIDIA H100**. - In **under 30 seconds**, Lightning AI moves your workspace over to an active 80 GB H100 node—keeping all your files, open terminal tabs, and code completely intact. - When training finishes, you switch back to Free CPU. You pay only for the exact minutes the GPU was running.”

Slide 11 (Distributed AI: DDP vs FSDP)

“When training larger models across multiple GPUs, engineers use two common techniques:

1. DDP (Distributed Data Parallel):

- The entire model is copied onto every GPU.
- Each GPU takes a different chunk of data and computes updates.
- At the end of each step, the GPUs sync their results over fast NVLink lines.
- This is fast and simple, but the entire model must fit inside a single 80 GB GPU.

2. FSDP (Fully Sharded Data Parallel):

- When a model has 70B or 400B parameters, it is too big for a single 80 GB card.
- FSDP splits the model weights, gradients, and optimizer across all GPUs in the cluster.

- Each GPU holds only a fraction of the model in memory. Layers are fetched right when needed over fast NVLink lines and then freed immediately.
- This lets teams train massive models without running out of memory.

In Lightning AI, you can scale from one GPU to multi-GPU FSDP without rewriting your code.

Now, let us see the real-world proof. I hand over to Ameer Hamza for our live demo."

(Hand off to Ameer Hamza)

SPEAKER PACKET 5: HARSHIT TANDON

Role: Cloud Economics, Hyperscaler Quotas & Enterprise TCO

Assigned Slides: Slide 14, Slide 15

Total Speaking Time: 5.0 Minutes (36:00 – 41:00)

Slide 14 (The Hyperscaler Bottleneck vs Lightning AI)

(Take over from Ameer Hamza)

“Thank you, Ameer. Let us look at the business and pricing side: **How you actually get access to an H100.**

If you try to rent an H100 on traditional clouds like **Amazon Web Services (AWS)** or **Google Cloud Platform (GCP)**, you immediately run into the **Quota Bottleneck**:

1. **Default Zero Limit:** Your account starts with zero GPU quota. You cannot launch a machine without submitting justification requests.
2. **Forced 8-GPU Bundles:** AWS and GCP do not rent single H100s. They force you to rent an entire 8-GPU box at **\$88 to \$98 per hour**.
3. **Long Contracts:** Sales teams often require 1-year or 3-year commitments of \$50,000 to \$100,000 upfront.
4. **Delays:** For students or small teams, approval takes days or weeks and is often rejected.

Lightning AI makes access simple: – You can rent a **single H100 GPU for about \$3.00 an hour**. – There are no sales calls, no quota forms, and no annual contracts. – Any student or startup can start an H100 in 30 seconds.”

Slide 15 (Enterprise TCO & Cost Optimization)

“Now look at the total cost comparison on Slide 15:

- **The Wasteful Cloud Approach:** A team starts an 8-GPU node on AWS and leaves it running all month. That costs **\$21,600 per month**. Studies show up to 60% of that GPU time sits completely idle while developers write code or fix bugs.
- **The Smart Studio Approach:** With Lightning AI, you match the hardware to the task:
 - Writing code and testing on **Free CPU: \$0.00**.
 - Training on an H100 for 10 minutes: **\$0.50**.
 - Keeping files in cloud storage: **a few cents per month**.

Notice the live demo Ameer just ran: – All four live tests ran in **less than 3 minutes of GPU time**. – The total bill was **under 15 cents—about 12 Indian Rupees!**

This turns AI supercomputing into an affordable service that any student or startup can use.

I invite Ameer back to wrap up our conclusions and start our viva.”

(Hand off to Ameer Hamza)

PART III: FACULTY VIVA DEFENSE — QUESTIONS & DIRECT ANSWERS

Question 1 (For Harsh Mishra):

Faculty Question: *"Why is FP8 training stable? Doesn't cutting numbers down to 8 bits cause gradient underflow and ruin accuracy?"*

Answer (Harsh Mishra):

"Standard 8-bit quantization would indeed cause underflow because 8 bits has a very narrow number range. Hopper solves this through the hardware **Transformer Engine** using dynamic scaling and dual-format switching: 1. It uses **E4M3** (more precision) during the forward pass where activation detail is critical. 2. It switches to **E5M2** (wider dynamic range) during the backward pass where gradient numbers become very small. 3. The hardware analyzes tensor values and computes dynamic scaling factors at every step so numbers stay inside range. This produces the exact same accuracy curve as FP16, but with double the compute throughput."

Question 2 (For Harsh Mishra):

Faculty Question: *"What is the physical difference between HBM3 and GDDR6? Why is HBM3 so much faster?"*

Answer (Harsh Mishra):

"It comes down to bus width and 3D stacking: - GDDR6 chips are soldered flat around the GPU die across a circuit board. Electrical traces limit the bus width to 128 or 256 bits, reaching around 272 to 500 GB/s. - HBM3 stacks memory dies vertically in 3D directly next to the GPU die on a silicon interposer, connected by thousands of tiny vertical pins called Through-Silicon Vias. - This creates a massive **5,120-bit wide memory bus**. Because the highway is 20 to 40 times wider, HBM3 transfers **3.35 Terabytes per second** while running at lower clock speeds and consuming less power per bit."

Question 3 (For Mirza Saad Beg):

Faculty Question: *"Explain the difference between DDP and FSDP. When would you choose FSDP over DDP?"*

Answer (Mirza Saad Beg):

"Both are distributed training methods in PyTorch: - **DDP (Distributed Data Parallel)** copies the entire model onto every GPU. Each GPU computes updates on its own data chunk and syncs gradients via All-Reduce. DDP is fast and simple, but the entire model must fit inside a single GPU (< 80 GB). - **FSDP (Fully Sharded Data Parallel)** splits model weights, gradients, and optimizer states across all GPUs. Each GPU holds only 1/Nth of the model in memory. During forward and backward passes, layer weights are fetched over NVLink right when needed and freed immediately. - **Rule of Thumb:** We use DDP for models up to ~13B parameters that easily fit in 80 GB. We switch to FSDP for 70B+ parameter models that exceed single-GPU memory."

Question 4 (For Suhail Alam):

Faculty Question: *"Why can't we parallelize deep learning by running multi-threading across a 64-core server CPU?"*

Answer (Suhail Alam):

"Because of physical hardware architecture and memory bandwidth: - A 64-core server CPU has only a few hundred vector execution units (AVX-512) and is limited by DDR5 memory bandwidth of about 200 to 300 GB/s. - An H100 GPU has **16,896 CUDA cores and 528 Tensor Cores** backed by 3,350 GB/s of HBM3 bandwidth. - Matrix math in transformers requires trillions of simultaneous multiply-accumulate operations. A CPU simply lacks the physical execution lanes and memory pipes to process that math without choking."

Question 5 (For Harshit Tandon):

Faculty Question: *"If AWS and Google Cloud offer enterprise discounts, why would a company choose Lightning AI?"*

Answer (Harshit Tandon):

"Because of Total Cost of Ownership and idle time: - Hyperscaler discounts require 1 to 3-year contracts. If your team's GPUs sit idle at night or over weekends, you still pay for every unused hour. - Also, on AWS you have to hire devops engineers costing over \$150,000 a year just to manage Kubernetes, clusters, and drivers. - Lightning AI lets teams develop on free CPUs and burst to an H100 strictly for the exact minutes needed, saving up to 80% on compute bills with zero devops overhead."

Question 6 (For Ameer Hamza):

Faculty Question: *"In Experiment 2, why did increasing the batch size from 1 to 16 on the H100 give a 27.8x speedup instead of just 16x?"*

Answer (Ameer Hamza):

"Because at Batch Size 1, the H100 is **under-utilized and memory-bandwidth bound**: - At batch size 1, the GPU spends most of its time reading weights from memory for just one sample, leaving thousands of Tensor Core execution lanes idle. - Increasing the batch size to 16 increases **arithmetic intensity**—meaning the GPU does much more math for every byte it loads. - The H100's 132 SMs become fully saturated. Step time only increased slightly from ~80ms to ~110ms, while sample throughput jumped from 5.4 to **147.1 samples per second**, giving us an empirical **27.8x speedup**."